AUGUSTE: Online-Learning dApp for Predictive URLLC Scheduling

Maxime Elkael†, Michele Polese†, Yunseong Lee‡, Koichiro Furueda‡, Tommaso Melodia†

arXiv:2606.03664v1 [cs.NI] 2 Jun 2026

***Abstract*—Ultra Reliable and Low Latency Communications (URLLC) was one of the main motivations behind 5G, with 3GPP advertising 1–10 ms latency targets for applications such as industrial automation, Vehicle-To-Everything (V2X), tactical edge networking, and unmanned-system control. Years on, real 5G Time Division Duplexing (TDD) networks still show median Uplink (UL) round-trip times in the 50–70 ms range, largely because of the Scheduling Request (SR) procedure that a User Equipment (UE) must complete before transmitting UL data. Existing remedies, primarily Configured Grant (CG) scheduling, only eliminate this overhead for strictly periodic traffic and require cross-layer synchronization, which has limited their adoption. We propose AUGUSTE (*Anticipatory Uplink Grants for URLLC via Self-Adapting Temporal Estimation*), a learning-based Medium Access Control (MAC) scheduling framework that embeds online Machine Learning (ML) models in the UL scheduler to predict packet arrivals and proactively allocate resources before an SR is issued. An adaptive state machine alternates between a *learning* phase that collects unbiased arrival statistics and a *confident* phase that exploits the learned predictions to schedule only when traffic is expected. We evaluate AUGUSTE on a real 5G testbed running OpenAirInterface across three URLLC traffic patterns (request-response, ML edge inference, and periodic autonomous reporting), and show that it operates at the best achievable point on the latency–overhead trade-off: it matches always-on scheduling's median Round Trip Time (RTT) (~ 10 ms, halving the ~ 20 ms SR-based baseline) at roughly one-tenth its resource cost (7–10% overhead).**

I. Introduction

Recent counter-Unmanned Aerial Vehicle (UAV) operations have made low-latency tactical wireless networking a frontline requirement. First-Person View (FPV) UAVs have become a defining feature of contemporary drone warfare, with teleoperation requiring tight latency and jitter bounds to maintain control and detection of fast-moving aerial objects. Further, real-time Artificial Intelligence (AI) inference is driving low latency requirements for reporting from tactical edge sensors and UAVs. Finally, the U.S. Department of Defense (DoD) Replicator initiative aims to field thousands of attritable (i.e., expendable) autonomous systems [1]. We argue their effectiveness will depend on the network supporting mass, bursty, low-latency UL communications. These tactical demands fall in the URLLC service class (sub-millisecond to tens-of-millisecond latencies at $\geq$ 99.999% reliability, depending on use case [2], [3]), and the same budget is load-bearing for applications such as V2X, telemedicine with haptic feedback, and industrial automation. In practice, however, real 5G TDD deployments fall short on both fronts: large-scale measurements of commercial 5G standalone (SA) networks [4] report median RTTs in the 50–70 ms range with tails reaching 200 ms.

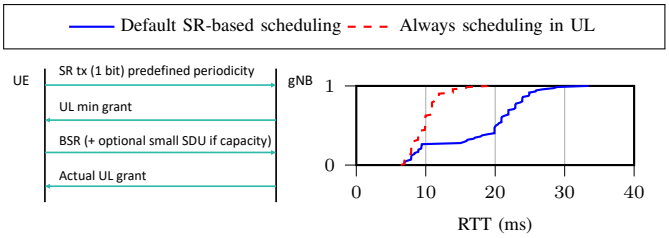


Fig. 1: SR procedure.

Fig. 2: RTT w/ & w/o always-on UL.

The main culprit on the UL is the SR procedure. As shown in Fig. 1, a UE with new UL data must first signal its intent to transmit by sending a SR on a periodically configured Physical Uplink Control Channel (PUCCH) opportunity; the Next Generation Node Base (gNB) then allocates a small grant, receives the Buffer Status Report (BSR), and finally schedules the actual data on a subsequent UL opportunity. For sparse traffic, this has to be done for every burst, with total overhead routinely exceeding 15 ms. The overhead is worst in TDD due to slot-alignment delays, hence, in this paper, we focus on TDD, but note the same SR procedure also applies in FDD.

To reduce this SR overhead, 3GPP introduced Configured Grant (CG) (or grant-free) scheduling [5], [6], in which the gNB pre-allocates periodic UL grants without a SR. This (i) only fits strictly periodic traffic, which is a poor match for event-driven (e.g., tactical sensing) or multi-source (e.g., a robot with multiple sensors) traffic, and (ii) imposes tight cross-layer synchronization between the Application and RRC Layer at the UE, complicating deployment and limiting adoption.

Beyond CG, ML-based scheduling has been studied at the MAC layer for Downlink (DL) resource allocation [7] and for URLLC/Enhanced Mobile Broadband (eMBB) mini-slot coexistence [8], and with O-RAN xApps for URLLC orchestration [9]. Cellular traffic prediction has also been studied extensively, but the bulk of this literature operates at coarser timescales (minutes to hours) and at cell-aggregate granularity [10]. AUGUSTE targets a different regime: per-slot UL grant decisions that bypass the SR procedure, via the *dApp* paradigm [11], i.e., with control logic deployed directly inside the Radio Access Network (RAN) node.

A simple workaround to reduce latency is to schedule

†Institute for Intelligent Networked Systems, Northeastern University (NU), Boston, MA, USA. E-mail: {m.elkael, m.polese, melodia}@northeastern.edu. ‡SoftBank Corp. (SB), Japan. E-mail: yunseong.lee.1992@ieee.org, koichiro.furueda@g.softbank.co.jp.

This work is partially supported by OUSD(R&E) through Army Research Laboratory Cooperative Agreement Number W911NF-24-2-0065. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the Army Research Laboratory or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein. This work is also partially supported by the U.S. NSF under award CNS-2112471 and by SoftBank Corp.

URLLC UEs proactively (i.e. without a SR or CG) in every slot (*always-on* scheduling), recently demonstrated in srsRAN as part of a broader latency-reduction effort [12]. This eliminates SR delay but wastes over 90% of the allocated radio resources for sparse traffic, while having poor scaling. AUGUSTE instead schedules *proactively but selectively*: it predicts when UL packets are about to arrive and pre-allocates resources only at those times. The predictor is trained *online*: its training data, the true UL arrival times, is generated by the framework's own actions rather than by an offline annotation pipeline. Because the SR mechanism biases reactive measurements, the framework cannot learn from passive observation; the proactive grants issued during a brief learning phase are what surface the un-distorted arrival times, which the predictor then consumes as training data. No offline dataset or operator annotation is required.

Our Contributions. (i) The first UL scheduling dApp that learns online from multiple in-RAN events (UL receptions, DL transmissions, and proactive-grant failures) to predict packet arrivals and bypass the SR procedure on a real 5G SA platform. (ii) An adaptive three-state (Learning/Confident/Idle) supervisor that trades off radio overhead and latency without Radio Resource Control (RRC) reconfiguration or prior knowledge of the traffic distribution, parameterized by two knobs that expose the latency-overhead trade-off explicitly. (iii) An end-to-end implementation of AUGUSTE as a custom OpenAirInterface (OAI) scheduler dApp, evaluated empirically on a real 5G SA testbed across three representative URLLC traffic patterns. We measure a 50% median RTT reduction (20 ms to 10 ms) at 7–10% radio overhead, and show that AUGUSTE operates at the best achievable point on this trade-off.

The rest of the paper is organized as follows. Section II formalizes the latency–overhead trade-off; Section III describes the proposed framework; Section IV details the experimental setup; Section V presents results; and Section VI concludes the article.

II. Problem Formulation

The UL scheduling problem in 5G URLLC is a trade-off between latency and resource efficiency. This section quantifies the SR overhead, formalizes the multi-objective trade-off, and identifies the resulting requirements for an effective solution.

A. Quantifying the Scheduling Request Overhead

Beyond the qualitative picture of Fig. 1, the SR procedure contributes several compounding delays. The UE first waits for its next SR opportunity. This comes with a delay ranging from 0 to the configured SR periodicity, typically 10–40 ms and up to 640 ms in 5G. Limited PUCCH resources force operators to use sparser periodicities as the number of UEs grows. Then, the SR is followed by a Downlink Control Information (DCI) grant on the next DL opportunity and UL transmission on the next UL slot (after at least one full TDD period). In this, a small initial grant typically carries only the BSR, requiring yet another TDD period for the actual data. Even in the best case, the total overhead is at least the SR periodicity plus one TDD period, and in practice ranges from 15–50 ms, far exceeding the 1–10 ms URLLC targets. The SR procedure is also a major source of jitter: the wait time depends on when a packet arrives relative to the SR periodicity, producing a near-uniform random delay that is damaging for applications demanding deterministic latency bounds.

Fig. 2 quantifies this overhead on our OAI testbed under no competing load (representative of a lightly-loaded cell or a URLLC slice with reserved resources). The default SR-based scheduling exhibits RTT values in the 15–25 ms range with median $\approx$ 20 ms and significant variance: already below commercial measurements but still well above URLLC targets. In contrast, when the UL scheduler systematically allocates resources without waiting for an SR ("always scheduling"), the RTT drops to 7–12 ms with a much tighter distribution. This $\approx$50% latency reduction directly corresponds to eliminating the SR overhead and its associated variability.

B. The Resource Efficiency Dilemma

While always scheduling successfully reduces latency and jitter, it creates an unsustainable resource overhead. For sparse URLLC traffic patterns typical in industrial Internet of Things (IoT) (e.g., periodic sensor reports at 10-100Hz), this approach wastes over 90% of allocated resources. With N URLLC UEs, the system would require N times the resources, severely limiting scalability.

We formulate this as the multi-objective optimization problem (P-URLLC), seeking Pareto-optimal scheduling strategies that trade off three objectives: worst-case expected latency, worst-case jitter, and total slice resource consumption.

$$\begin{aligned} \underset{S\in\mathcal{S}}{\text{minimize}} \quad & \begin{bmatrix} \max_{i\in\mathcal{N}} \mathbb{E}[L^i_{sched}(S)] \\ \max_{i\in\mathcal{N}} V(L^i_{sched}(S)) \\ \sum_{i\in\mathcal{N}} U_i(S) \end{bmatrix} \\ \text{subject to} \quad & U_i(S) = \frac{\mathbb{E}[R_i(S)]}{R_{slice}/|\mathcal{N}|}, \quad \forall i \in \mathcal{N}, \end{aligned} \tag{P-URLLC}$$

where:

- $\mathcal{N}$ is the set of UEs in a URLLC slice, assumed to have homogeneous Quality of Service (QoS) requirements
- $S \in \mathcal{S}$ is a feasible scheduling strategy
- $L^i_{sched}(S)$ is the scheduling-induced latency for UE i
- $\mathbb{E}[L^i_{sched}(S)]$ is the expected scheduling latency for UE i
- $V(L^i_{sched}(S))$ is the latency variance (jitter) for UE i
- $U_i(S)$ is the resource consumption ratio for UE i
- $R_i(S)$ is the random variable representing instantaneous resources allocated to UE i
- $\mathbb{E}[R_i(S)]$ is the time-averaged resources allocated to UE i
- R_{slice} is the total Physical Resource Block (PRB) resources allocated to this URLLC slice
- $|\mathcal{N}|$ is the number of UEs in the slice

The three objectives reflect different aspects of URLLC requirements within a slice: the first ensures no individual UE

experiences excessive expected latency (fairness and worst-case guarantee), the second ensures predictable performance by minimizing the worst-case jitter across all UEs, and the third minimizes total slice resource consumption for scalability. The expectations and variances are taken over the stochastic packet arrival process and channel variations.

This problem is challenging because $L^i_{sched}(S)$ and $U_i(S)$ are inversely coupled: strategies that minimize latency (e.g., always-on scheduling) maximize resource usage, while strategies that minimize overhead (e.g., SR-based scheduling) maximize latency. The discrete nature of PRB allocation and the stochastic arrival patterns of traffic make (P-URLLC) non-convex, with no closed-form solution. Standard approaches select fixed points on the trade-off curve: SR-based scheduling chooses minimum resources at the cost of high latency, always-on scheduling chooses minimum latency at the cost of maximum resources, and CG scheduling attempts a middle ground for the restricted case of strictly periodic traffic. None of these can adapt their operating point to the actual arrival process, nor can they exploit the temporal correlations in URLLC traffic to improve both objectives simultaneously.

III. System Design

Solving (P-URLLC) exactly is intractable: the latency random variable $L^i_{sched}(S)$ depends on the unknown UL arrival process, and no closed-form Pareto frontier exists. We instead propose an online strategy S_{ML} that (i) estimates the arrival process from in-band scheduler events using lightweight ML models embedded directly in the MAC scheduler as a Layer-2 dApp, and (ii) lets the operator tune its position on the latency-overhead trade-off through two knobs, a tolerance window TW and a slot restriction N, introduced in Section III-C. Section V shows that the learned predictions are accurate enough to push S_{ML} close to simultaneous minimum overhead and minimum latency. The framework relies on an adaptive state machine that alternates between aggressive learning phases that collect unbiased arrival statistics and prediction-driven proactive scheduling phases that exploit them. Before describing the architecture, we first show that even a primitive instance of S_{ML} already approaches always-on latency at a fraction of its overhead.

A. Proof of Concept

Fig. 3 measures RTT in a real 5G network where a UE receives periodic pings from the User Plane Function (UPF), and where the scheduler proactively allocates UL resources whenever a DL transmission occurred within a lookback of 1–4 TDD periods. With 2 periods the median RTT drops to ∼9 ms (i.e., comparable to always-on scheduling) while resource overhead is significantly reduced. A coarse correlation signal (here, recent DL transmissions) is therefore already informative enough to bypass the SR; the rest of this section generalizes this idea into the full learning framework.

B. System Architecture

AUGUSTE follows a Callback (CB)-based architecture (Fig. 4): MAC-layer events from the scheduler are surfaced

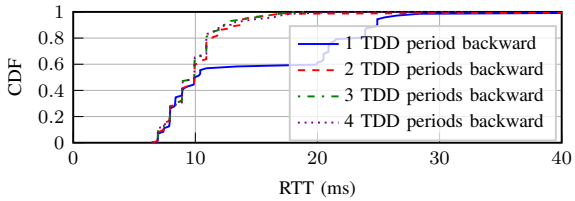


Fig. 3: RTT w/ DL-aware proactive scheduler at varying lookback windows.

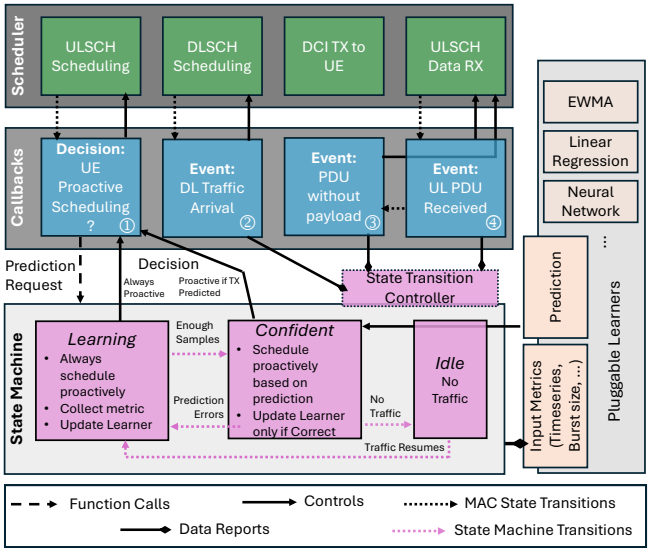


Fig. 4: Proposed AUGUSTE scheduler framework.

through CBs that feed a state machine, which then modifies the Uplink Shared Channel (ULSCH) scheduling. AUGUSTE instruments three ULSCH CBs. In Fig. 4, CB ① is a per-slot proactive query to the state machine, CB ④ reports Radio Network Temporary Identifier (RNTI), Service Data Unit (SDU) size, timestamp, and grant type for each received Packet Data Unit (PDU), CB ③ fires when a proactive grant returned empty, and CB ② is the only one that happens in the Downlink Shared Channel (DLSCH); it reports timestamp and size of each DL transmission. Additional CBs (e.g., pilot or Hybrid Automatic Repeat reQuest (HARQ) reception) can be added without modifying the rest of the framework; the four above suffice for our use cases.

We implement AUGUSTE as a set of modifications to the OAI gNB MAC layer. The underlying allocation policy keeps the Proportional Fair (PF) algorithm [13]. AUGUSTE's control CB ① hooks into the UL scheduling loop and inserts the target UE into the candidate set of the current slot whenever the state machine triggers a proactive grant. Because the UE has not issued an SR, no real BSR is available, so the CB also attaches a virtual non-zero buffer size for the PF scheduler to size the grant. The scheduler then allocates PRBs as it would for any UE with pending data, so Modulation and Coding Scheme (MCS) selection, link adaptation, and inter-UE fairness behave identically to the baseline. Proactive scheduling changes *who* is eligible in a given slot, not *how* allocation happens, so the framework composes with any underlying scheduling policy. Under PF specifically, the two

mechanisms reinforce each other: the PF metric is the ratio of instantaneous achievable rate to historical throughput, and sparse URLLC traffic drives the denominator toward zero, so a proactively scheduled UE carries an effectively unbounded PF score and dominates contention against any non-URLLC UE. Our experiments assume no congestion, which does not always hold in practice; the standard remedy is to group UEs and reserve a fraction of the cell's PRBs for them (i.e., slicing), which we have previously enabled in [13].

C. State Machine Design

A state machine instance is associated with each group of UEs sharing a common arrival distribution: per-UE when no such prior is available, or per-group when UEs are known to share statistics (e.g., identical sensors on a fixed reporting schedule). We describe one such instance below; the framework runs them independently, one per group.

At initialization the instance is in the *Learning* state, where it schedules proactively at every slot. This both masks the SR delay and collects an unbiased dataset of true UL arrival times. In parallel, the learner is updated; it can be any ML model (e.g., Exponentially Weighted Moving Average (EWMA), linear regression, pretrained Neural Network (NN)) that consumes any subset of scheduler-observable features (past UL inter-arrival times, DL event sizes and timestamps, DL-to-UL response times, traffic class) and emits a predicted slot offset for the next UL arrival. The feature subset is deployment-specific: periodic reporting may use only past inter-arrival times, while cued edge inference exploits recent DL burst sizes. The transition to *Confident* fires once a learner-specific readiness condition is met (e.g., N samples collected or prediction variance below threshold).

In the *Confident* state, scheduling follows the learner's prediction, governed by two operator-tunable knobs: a tolerance window TW that admits scheduling within $\pm$TW TDD periods of the prediction, and a slot restriction N that limits proactive grants to the last N slots of each TDD period. Smaller (TW, N) minimize overhead by committing to the predicted slot only; larger values widen the safety window at the cost of additional grants. In the language of (P-URLLC), the *Learning* state realizes the $\min\max_i \mathbb{E}[L^i_{sched}]$ extreme at the cost of maximal $\sum_i U_i$, while $(\mathrm{TW}, N) = (0, 1)$ approaches the opposite extreme (minimum overhead, latency bounded by prediction error). Intermediate values trace the empirical Pareto frontier; the achievable trade-off depends entirely on prediction accuracy.

Prediction quality is monitored online via two counters: false negatives (reactive transmissions, tracked as consecutive misses) and false positives (unused proactive grants, tracked cumulatively). Crossing configurable thresholds (e.g., 3 consecutive misses or a 10% false-positive rate) triggers a state transition. The framework also supports learners that emit (TW, N) dynamically, letting the system absorb drift in the arrival distribution by temporarily widening its safety window without fully reverting to *Learning*. A third *Idle* state handles UEs with extended inactivity (no UL for 100+ slots), pausing prediction until traffic resumes.

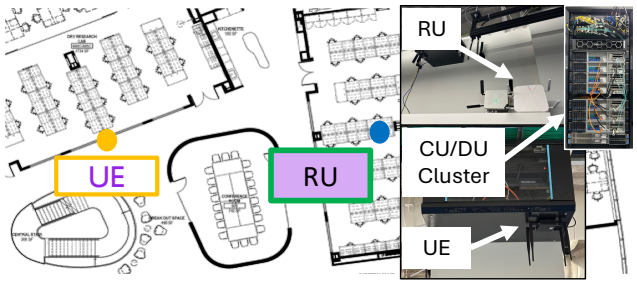


Fig. 5: Lab testbed for AUGUSTE evaluation.

Together, the three states let the framework balance latency, jitter, and overhead under unknown and possibly drifting traffic, without RRC reconfiguration or a priori knowledge of the arrival process.

Scaling Considerations. The framework's scaling is bounded primarily by Physical Uplink Shared Channel (PUSCH) PRB consumption per UL slot. Each proactive grant consumes a tunable number of PRBs sized to the expected payload, so the per-slot PRB budget caps the number of concurrent proactive UEs. A 100 MHz cell carries up to 273 PRBs per slot; even the extreme lower bound of 1 PRB per grant yields a Transport Block Size (TBS) of only a few tens of bytes at typical channel conditions [14]. The Physical Downlink Control Channel (PDCCH), which carries the grants themselves, is not the binding constraint: a typical CORESET configuration can schedule tens of UEs per slot [15], and typical TDD patterns have more DL slots (which carry grants) than UL slots (which carry data).

IV. Experimental Setup

We evaluate AUGUSTE on a real OAI-based 5G SA over-the-air deployment built on the X5G testbed [16] and the AutoRAN framework [17]. The gNB runs on a Dell PowerEdge R760 server connected to a Foxconn RPQN Radio Unit (RU) operating in band n78 with 100 MHz bandwidth and numerology $\mu = 1$. A single Sierra Wireless EM9293 UE communicates over the air in a lab environment with fair channel conditions (median Channel Quality Information (CQI) 13). The network operates with a TDD pattern of DDDDDDDSUU (10-slot periodicity) typical of commercial deployments. Fig. 5 summarizes the deployment.

A. Evaluation Scenarios

We design three complementary scenarios that represent key URLLC use cases where proactive scheduling provides significant benefits.

Request-Response Pattern (RTT Measurements). We generate ICMP echo requests (300-byte payload) from the UPF to the UE at controlled intervals. This scenario represents query-response applications where network elements request data from edge devices, e.g., command-and-control polling of forward assets, FPV UAV teleoperation control loops, and analogous civilian workloads such as industrial sensors responding to polling or V2X infrastructure querying

vehicle status. The predictable DL-to-UL pattern allows us to evaluate RTT-based learning where the scheduler predicts UL responses based on prior DL transmissions. In this scenario, the framework is configured to use an EWMA learner.

Cued Edge Inference. We emulate workloads where a central node queries an edge device, which runs local ML inference and returns a variable-size result. This pattern appears in tactical Intelligence, Surveillance, and Reconnaissance (ISR) (cued sensor exploitation, where a forward sensor is tasked to run a focused classifier on a target region), tactical Natural Language Processing (NLP)/signal intelligence triage (forward transcription or translation of pushed audio), and edge-cloud split AI assistants on forward operating bases. The common shape across these use cases is that the inference latency and the size of the resulting UL payload both depend on characteristics of the incoming DL traffic. We instantiate this scenario by running an Arctic Embed XS text-embedding model on the UE: it receives a variable-length prompt over DL, performs inference whose duration depends on input length, and returns the embedding over UL, with the server at the UE and the client at the UPF communicating over User Datagram Protocol (UDP). We vary the input by generating random word sequences of different lengths, which translates into different packet sizes at the MAC layer. AUGUSTE is configured to use a Linear Regression on the DL burst size for prediction; this is one natural instantiation of the more general principle that the learner can consume any scheduler-observable feature correlated with the UL arrival time.

Periodic Autonomous Reporting. We generate periodic UL-only traffic at fixed intervals (10-100 ms) with no preceding DL trigger, representing workloads such as unattended tactical sensors, periodic status reports from autonomous systems, mission-critical telemetry, and analogous industrial IoT traffic (heartbeat messages, autonomous sensor reporting). This scenario has no DL correlation to exploit, requiring the scheduler to learn pure inter-arrival patterns. It demonstrates the framework's ability to handle traffic that grant-free scheduling targets but without the associated rigidity constraints. For this experiment, we use mgen to send 97 packets per second. We select this value because it is a prime number, which ensures that the position of packets in the TDD period does not repeat itself; it also produces a 10.31 ms inter-arrival time that falls between the standardized CG periodicities (10 and 20 ms), making this scenario representative of periodic workloads that CG scheduling cannot serve without phase drift. In this scenario, we use an EWMA learner.

For evaluation, we disable the Confident-to-Learning fallback which would normally be triggered by accumulated prediction errors (Sec. III-C). Otherwise, measured latency and overhead would mix two regimes: the predictor's accuracy in the Confident state, and the always-schedule mode of the Learning state, which by construction sits at the (100% overhead, minimum delay) corner. Disabling the fallback isolates the first effect, so the reported numbers reflect Confident-state prediction quality alone. Our baseline for evaluation is the OAI scheduler which does not perform proactive scheduling.

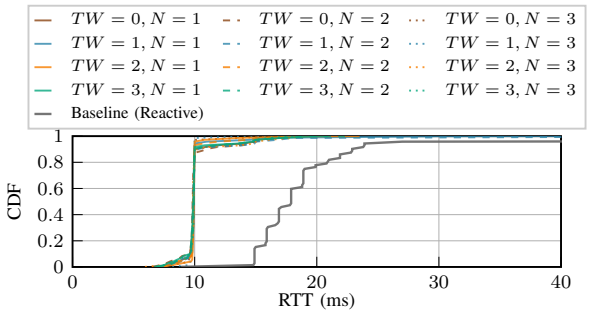


Fig. 6: RTT for the request-response scenario.

V. Performance Evaluation

For each scenario we evaluate the latency CDFs across the (TW, N) sweep, together with the overhead (i.e., the fraction of slots in which the UE is scheduled) for the request-response case. The sweep traces an empirical latency-overhead trade-off curve, and two observations recur across all three scenarios. First, the curve exhibits a sharp *knee*: a single (TW, N) configuration achieves near-minimum overhead and near-minimum latency simultaneously, while more aggressive configurations are dominated (i.e., gain nothing on either metric). Second, the location of the knee is an empirical readout of prediction quality. For example, a knee at $(\mathrm{TW}, N) = (0, 1)$ means the learner localizes UL arrivals to within a single TDD period, so larger safety windows offer no further benefit.

Request-Response (RTT). We vary TW between 0 (proactive scheduling only if the TDD period matches the prediction exactly) and 3 (± 3 periods around the prediction). We vary N between 1 and 3, since we have at most 3 UL slots per TDD period in our pattern (two U slots and one mixed S slot).

Fig. 6 presents the latency results. For every (TW, N), the latency distribution improves substantially over the baseline: the median drops from ~ 20 ms with a long tail to 10 ms, with a 90th percentile also at 10 ms and no long tail.

As depicted in Fig. 7, the overhead (measured as the proportion of UL slots with some resources scheduled, either proactively or reactively) scales linearly with TW and N. Hence, the optimal configuration for this scenario is $\mathrm{TW} = 0$, $N = 1$, where the overhead is 7%. On the other hand, the baseline has the lowest overhead with 4% of slots scheduled. While we present detailed overhead analysis only for the request-response scenario, the overhead scales similarly with TW and N across all scenarios, with absolute values proportional to the traffic rate; we omit the overhead plots for the other scenarios for brevity.

Cued Edge Inference. We use $\mathrm{TW} = 2$ and $N = 1$. As shown in Fig. 8, the scheduler predicts the UL arrival time from the DL burst size, giving an end-to-end delay reduction of ~ 10 ms over the baseline across all input sizes, in line with the network latency saved in the request-response scenario.

Periodic Autonomous Reporting. Fig. 9 presents the third scenario. As before, we vary TW and N between 0 and 3. $\mathrm{TW} = 0$ already improves the CDF over the baseline, but the best results appear from $\mathrm{TW} = 1$, with a median one-

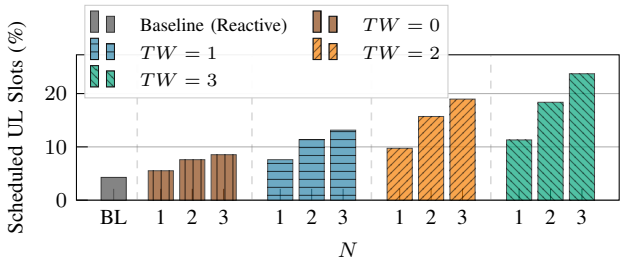


Fig. 7: Resource overhead for the request-response scenario; bottom numbers indicate the slot restriction N.

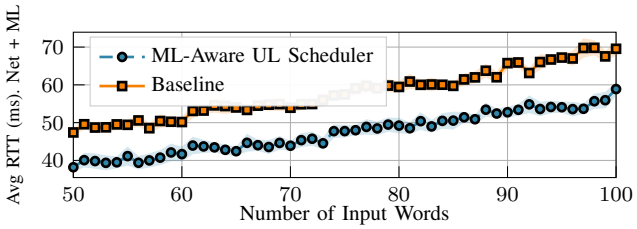


Fig. 8: E2E latency for the cued edge inference scenario.

way latency of 7 ms and a 99th percentile of 10 ms. The shift in optimal TW reveals a difference in detected traffic pattern for this use case. The prime 97-packet/s rate produces a 10.31 ms inter-arrival that is not a multiple of the TDD period, so successive packets walk across slot boundaries within the pattern. A predictor with $\mathrm{TW} = 0$ commits to a single TDD period and misses the arrival as drift accumulates; $\mathrm{TW} = 1$ absorbs the drift at low overhead. CG cannot apply the same optimization: its fixed periodicities (10 and 20 ms) lock the grant to a single phase, so the same drift that $\mathrm{TW} = 1$ tolerates would steadily desynchronize a CG schedule from the actual arrivals. The difference of up to a tenth of a millisecond between $N = 1, 2, 3$ comes from the high regularity of the periodic generation, where scheduling a few slots earlier has a small but noticeable impact on UL latency. The optimal operating point on the Pareto frontier is hence traffic-dependent; as real traffic is dynamic, AUGUSTE supports learners that emit (TW, N) from observed statistics.

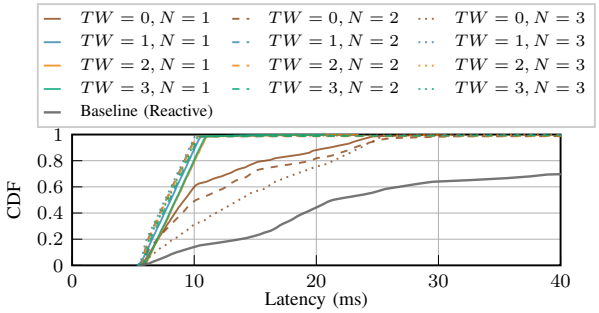


Fig. 9: One way latency for the uplink autonomous reporting.

VI. Conclusions

This paper presented AUGUSTE, a learning-based MAC scheduling framework that addresses the latency–efficiency trade-off in 5G URLLC systems. By embedding ML models in the UL scheduler and supervising them with an adaptive state machine, we eliminate the SR-induced delay and keep radio overhead close to the SR-based baseline. The integration with OAI leaves the underlying PF allocation policy unchanged, so the framework only modifies which UEs are eligible in a given slot and composes with any scheduling policy.

Across three URLLC scenarios on a 5G testbed, AUGUSTE reduces median RTT by $\approx 50\%$ at 7–10% radio overhead, and reduces median one-way latency from ~ 13 ms to 7–10 ms. The empirical (TW, N) sweeps trace a Pareto frontier of the latency–overhead trade-off with a sharp knee, showing that the learner localizes UL arrivals accurately enough that larger safety windows offer no further benefit. AUGUSTE therefore operates at the best achievable point on this trade-off.

Future work includes multi-UE experiments under congestion, online adaptation of (TW, N), and more expressive learners for traffic with multiple superimposed periodicities. On the channel side, each grant commits an MCS up to TW periods ahead, so we plan to predict channel evolution from sparse feedback (Sounding Reference Signal (SRS) / Channel State Information (CSI)). Finally, the implementation-level optimizations of [12] are orthogonal to AUGUSTE: they eliminate the SR via static per-slot reservation (the always-on extreme of our Pareto frontier), whereas AUGUSTE eliminates it adaptively at $\sim$10% overhead. Combining the two would compound the lower-level latency reductions with our prediction-driven overhead savings.

References

[1] Congressional Research Service, "DOD Replicator Initiative: Background and Issues for Congress," Tech. Rep., 2024.

[2] 3GPP, "Service requirements for the 5G system; Stage 1," 3GPP, TS 22.261, 2024.

[3] P. Popovski, J. J. Nielsen, C. Stefanovic, E. de Carvalho *et al.*, "Wireless Access for Ultra-Reliable Low-Latency Communication: Principles and Building Blocks," *IEEE Network*, vol. 32, no. 2, pp. 16–23, 2018.

[4] M. Ghoshal, O. Basit, S. Wang, P. Dinh *et al.*, "A First Large-Scale Study of Operational 5G Standalone Networks," *CoNEXT*, 2025.

[5] 3GPP, "NR; MAC protocol specification," TS 38.321, 2024.

[6] T.-K. Le, U. Salim, and F. Kaltenberger, "Enhancing URLLC Uplink Configured-grant Transmissions," in *ICC*, 2021.

[7] F. AL-Tam, N. Correia, and J. Rodriguez, "Learn to Schedule (LEASCH): A Deep Reinforcement Learning Approach for Radio Resource Scheduling in the 5G MAC Layer," *IEEE Access*, 2020.

[8] A. Anand, G. de Veciana, and S. Shakkottai, "Joint Scheduling of URLLC and eMBB Traffic in 5G Wireless Networks," *ToN*, 2020.

[9] R. M. Sohaib, S. T. Shah, O. Onireti, Y. Sambo *et al.*, "DRL-based Joint Resource Scheduling of eMBB and URLLC in O-RAN," in *ICC*, 2024.

[10] W. Jiang, "Cellular traffic prediction with machine learning: A survey," *Expert Syst. Appl.*, vol. 201, p. 117163, 2022.

[11] A. Lacava, L. Bonati, N. Mohamadi, R. Gangula *et al.*, "dApps: Enabling Real-Time AI-Based O-RAN Control," *Comput. Net.*, 2025.

[12] A. Gong, A. Maghsoudnia, R. Cannatà, E. Vlad *et al.*, "Towards URLLC with Open-Source 5G Software," in *OpenRIT6G*, 2025.

[13] M. Elkael, M. Polese, R. Prasad, S. Maxenti *et al.*, "ALLSTaR: Automated LLM-driven scheduler generation and testing for intent-based RAN," in *http://arxiv.org/abs/2505.18389*, 2025.

[14] 3GPP, "NR; PHY layer procedures for data," 3GPP, TS 38.214, 2024.

[15] M. Mozaffari, Y.-P. E. Wang, and K. Kittichokechai, "Blocking Probability Analysis for 5G New Radio (NR) Physical Downlink Control Channel," in *ICC*, 2021.

[16] D. Villa, I. Khan, F. Kaltenberger, N. Hedberg *et al.*, "X5G: An Open, Programmable, Multi-vendor, End-to-end, Private 5G O-RAN Testbed with NVIDIA ARC and OpenAirInterface," *Trans. Mobile Comput.*, 2025.

[17] S. Maxenti, R. Shirkhani, M. Elkael, L. Bonati *et al.*, "AutoRAN: Automated and Zero-Touch O-RAN sys." *Trans. Mobile Comput.*, 2026.